I102 - Paradigmas de Programación

OOP - HERENCIA

6

Prof. Mariano Scaramal

Ingeniería en Inteligencia Artificial

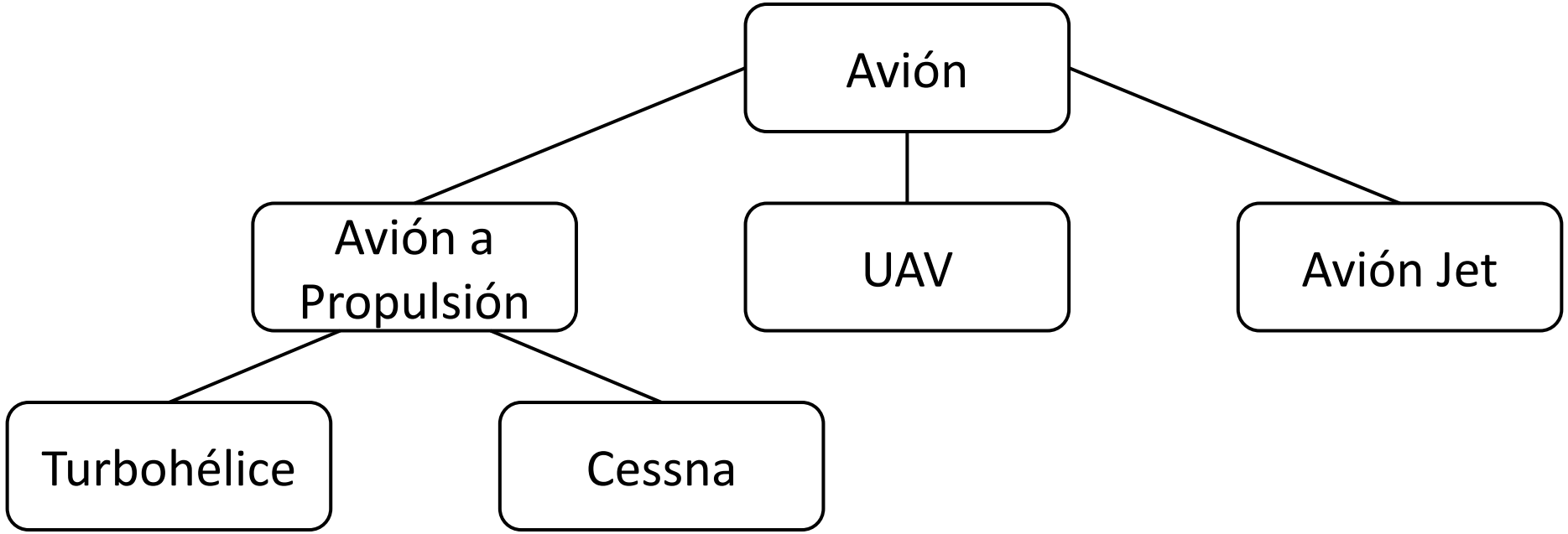
Esta Presentación Cubre…

- Conceptos:
  - Herencia:
    - Herencia Pública y Privada
    - Constructores y Destructores en Herencia
    - Herencia Múltiple
    - Ejemplo de Herencia
  - Tipos de relaciones entre objetos

Herencia

Herencia implica la creación de una nueva clase a partir de una clase ya existente de la cual hereda sus atributos y métodos. Esto es, la nueva clase, la *clase derivada,* hereda sus datos y funciones miembro de la *clase base*.

Conceptualmente las clases derivadas son más específicas que las clase base.



Herencia

```
#include <iostream>
#include <string>

class Avion {
protected: // No es ni público, ni privado! solo para herencia
    std::string modelo;

public:
    Avion(std::string m) : modelo(m) {}

    void despegar() {
        std::cout << "El avion " << modelo << " esta despegando..." << std::endl;
    }
};
```

Clase Base

Herencia

```
class AvionJet : public Avion { // Se dice que “AvionJet” hereda de “Avion”
private:
    int numTurbinas;

public:
    // El constructor de la hija DEBE llamar al de la madre
    AvionJet(std::string m, int t) : Avion(m), numTurbinas(t) {}
    void activarAfterburner() {
        std::cout << modelo << " activando postcombustion en sus" << numTurbinas << " turbinas!" << std::endl;
    }
};

class UAV : public Avion {
public:
    UAV(std::string m) : Avion(m) {}
    void transmitirVideo() {
        std::cout << "El UAV " << modelo << " transmitiendo video en tiempo real." << std::endl;
    }
};
```

Clases Derivadas

Herencia

```
int main() {
    AvionJet f16("F-16", 2);
    UAV uav("MQ-1");

    // Métodos heredados de la clase base
    f16.despegar();
    uav.despegar();

    // Métodos específicos de cada clase derivada
    f16.activarAfterburner();
    uav.transmitirVideo();

    return 0;
}
```

Main

Herencia (cont'd)

La herencia puede hacerse en forma pública (public), protegida (protected) o privada (private). Dependiendo de como sea, el acceso a las variables/funciones miembro será:

Encapsulamiento en la base	Herencia Pública	Herencia Protegida	Herencia Privada
public	public	protected	private
protected	protected	protected	private
private (default)	No accesible	No accesible	No accesible

Es muy importante recordar que los atributos y métodos privados siempre serán accesibles desde el mismo objeto y desde ningún otro lado más.

Al mismo tiempo, recuerde que si no se agrega ningún identificador de encapsulamiento, C++ entenderá que el mismo es <u>private</u> (default).

Ejemplo – Herencia Pública y Privada

Suponiendo que se tengan las siguientes clases base:

```
// Clase Base Avion (se usará como herencia pública)
class Avion {
public:
   Avion(std::string modelo) : modelo(modelo) {}

   void volar() const {
      std::cout << modelo << " esta volando!"
                << std::endl;
   }

protected:
   std::string modelo;
};
```

```
// Clase Base Motor (se usará como herencia privada)
class Motor {
public:
   Motor(float horsepower) : horsepower(horsepower) {}

   void encenderMotor() const {
      std::cout << "Encendiendo motor con "
                << horsepower << " HP." << std::endl;
   }

protected:
   float horsepower;
};
```

Ejemplo – Herencia Pública y Privada (cont'd)

Universidad de SanAndrés

Al crearse la base *PropPlane*, esta hereda los miembros públicos y protegidos de la clase *Avion* y *Motor*. No obstante, los miembros de *Motor* no pueden ser accedidos desde el objeto *cessna* en *main.*

```
// Clase deriveda PropPlane (herencia publica de Avion y
// privada de Motor)
class PropPlane : public Avion, private Motor {
public:
   PropPlane(std::string modelo, float horsepower)
      : Avion(modelo), Motor(horsepower) {}

   void encenderPropPlane() {
   // Está permitido llamar a encenderMotor() desde
   // PropPlane porque  Motor se hereda en forma privada
      Motor::encenderMotor();
      std::cout << modelo << " esta listo para despegar!" <<
      std::endl;
   }
protected:
  int nroPasajeros = 1;
};
```

Aquí podría usarse Avion::modelo, pero no hay ambigüedad.

```
int main() {
   PropPlane cessna("Cessna 172", 160);
   // Se llama un metodo de PropPlane
   cessna.encenderPropPlane();
   // Se llama este metodo heredado de Avion
   cessna.volar();
   // cessna.encenderMotor();  // Error: encenderMotor() es
   //privada en PropPlane porque se heredo de Motor en forma
   //privada (ver class PropPlane)
   // std::cout << cessna.nroPasajeros;  // Error: nroPasajeros no
   // se puede acceder!   Por qué?

   return 0;
}
```

Constructores y Destructores en Herencia

Una clase derivada siempre llamará el constructor de su clase base primero.

Los destructores son llamados en orden inverso, la clase derivada primero.

Si no existe un constructor default en la clase base, el compilador creará uno vacío para poder instanciar la clase base.

```
Creando objeto de clase Derivada
Constructor de clase base
Constructor de clase intermedia
Constructor de clase derivada
Fin del main.
Destructor de clase derivada
Destructor de clase intermedia
Destructor de clase base
```

```
#include <iostream>

class Base {
public:
   Base() { std::cout << "Constructor de clase base" << std::endl; }
   ~Base() { std::cout << "Destructor de clase base" << std::endl; }
};

class Intermedia : public Base {
public:
   Intermedia() { std::cout << "Constructor de clase intermedia" << std::endl; }
   ~Intermedia() { std::cout << "Destructor de clase intermedia" << std::endl; }
};

class Derivada : public Intermedia {
public:
   Derivada() { std::cout << "Constructor de clase derivada" << std::endl; }
   ~Derivada() { std::cout << "Destructor de clase derivada" << std::endl; }
};

int main() {
   std::cout << "Creando objeto de clase Derivada\n";
   Derivada obj;
   std::cout << "Fin del main.\n";
   return 0;
}
```

Constructores Explícitos e Implícitos en Herencia

- El constructor de la clase base puede ser llamado desde la clase derivada para evitar repetir código.
- Estos llamados pueden ser explícitos o implícitos (ver ejemplos en la diapositiva siguiente)
- En la forma explícita, el llamado se realiza al llamar el constructor de la clase base desde el constructor de la clase derivada, por lo tanto, el mismo debe escribirse.
- En la forma implícita, no es necesario escribir el constructor que llama al constructor de la clase base.
- Se pueden heredar múltiples constructores de la clase base, pero estos también pueden ser sobrescritos.

Llamadas Explícitas e Implícitas a Constructores

```
#include <iostream>

class Base {
public:
   Base(int x, int y) {  // Constructor de la clase Base
      std::cout << "El constructor base es llamado con x="
                << x << ", y=" << y << std::endl;
   }
};

class Derived : public Base {
public: // Se llama el constructor de Base explícitamente
   Derived(int xx, int yy) : Base(xx, yy) {
      std::cout << “Se llama al constructor derivado" << std::endl;
   }
};

int main() { // Se llama Base(10,20) con Derived(10,20)
   Derived obj(10, 20);
   return 0;
}
```

Llamada explícita al constructor base

```
#include <iostream>

class Base {
public:
   Base(int x, int y) {  // Constructor de la clase Base
      std::cout << "El constructor base es llamado con x="
                << x << ", y=" << y << std::endl;
   }
};

class Derived : public Base {
public: // Se heredan el/los constructor/es de Base
   using Base::Base; // Se usa cuando los constructores
                     //necesarios ya existen en Base
}; // Si se escribiera un constructor igual a alguno de Base,
se reemplazá. Si se pone uno nuevo, se agrega.

int main() {
   Derived obj(10, 20); // No hay constructor en Derived
   return 0;
}
```

Llamada implícita al constructor base

Llamadas Implícitas Múltiples a Constructores

```
#include <iostream>

class Base {
public:
   Base() {  // Constructor default de Base
      std::cout << "Constructor Default de Base\n";
   }

   Base(int x) {  // Constructor de Base con 1 param
      std::cout << "Constructor de Base con x=" << x
                << std::endl;
   }

   Base(int x, int y) {// Constructor de Base con 2 params
      std::cout << "Constructor de Base con x=" << x
                << " e y = " << y << std::endl;
   }
};
```

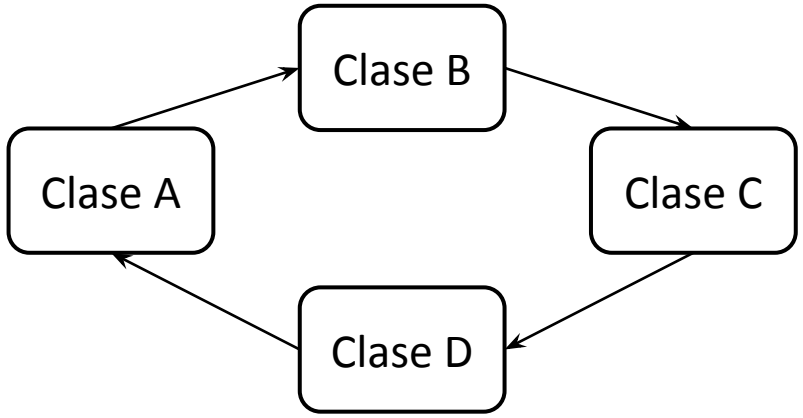
```
class Derivada : public Base {
public: // Se heredan todos los constructores de Base
   using Base::Base;

   Derivada(int x, int y) {
      std::cout << "Constructor de Derivada con x="
                << x << " e y=" << y << std::endl;
   } // Se sobreescribe el const de Base de 2 params
};

int main() {
   // Constructor default de Base
   Derivada obj1;
   // Constructor de 1 param de Base
   Derivada obj2(10);
   // Constructor sin parametros de Base y …
   // constructor con 2 params de Derivada
   Derivada obj3(10, 20);
   return 0;
}// :Base(x,y) para el contructor de Base de 2 params.
```

Herencia Múltiple

- Como se vio anteriormente, una clase puede tener varias clases base.
- De esta manera, una clase puede heredar múltiples comportamientos y datos.
- Si dos métodos de las clases bases poseen el mismo nombre, puede producir ambigüedad. En esto casos, siempre conviene usar las referencias a las clases: Base1::comer() o Base2::comer().
- Herencia múltiple puede conducir a ciclos cerrados de herencia que son ilegales en C++ (se producirá un error).



Herencia Múltiple – Problema del Diamante

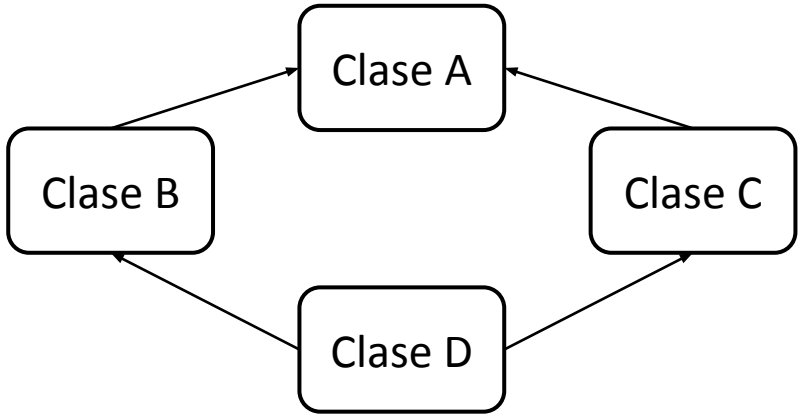
Otro problema que se puede generar con herencia múltiple es *el problema del diamante*, cuyo nombre proviene de las relaciones de herencia entre las clases.

Ocurre cuando hay varias clases base (clases B y C) que tienen la misma clase derivada (clase D) y una misma clase ancestro (clase A).

El problema aparece cuando se desea hacer uso de un método de la clase ancestro, para la cual existen dos posibles caminos (a través de la clase B y la C), generándose una ambigüedad.

Notar que el sentido de las flechas es distinto al caso anterior, y sólo indican el sentido de la herencia.

La clase A es la superclase de B y C, y estas son las clases base de D.



Problema del Diamante (en código)

```
#include <iostream>
using namespace std;

class A {
public:
   A() { cout << "Constructor de A" << endl; }
   void show() { cout << "Showing..." << endl; }
};

class B : public A {
public:
   B() { cout << "Constructor de B" << endl; }
};
```

```
class C : public A {
public:
   C() { cout << "Constructor de C" << endl; }
};

class D : public B, public C {
public:
   D() { cout << "Constructor de D" << endl; }
};

int main() {
   D d;
   // Hay dos caminos para llegar a show()
   d.show(); //Error: "D::show" is ambiguous

   return 0;
}
```

Problema del Diamante (en código)

```
#include <iostream>
using namespace std;

class A { public: int x; };
class B : public A { };
class C : public A { };
class D : public B, public C { };

int main() {
    D objeto;
    // objeto.x = 10; // Error por Ambigüedad. ¿B::x o C::x?
    // Para desambiguar: objeto.[ClasePadre]::[Atributo]
    objeto.B::x = 10; // Acceso manual a la copia de B
    objeto.C::x = 20; // Acceso manual a la copia de C

    std::cout << objeto.B::x; // Imprime 10
    std::cout << objeto.C::x; // Imprime 20 (variables distintas)
}
```

Cómo Evitar el Problema del Diamante

```
#include <iostream>
using namespace std;

class A {
public:
   A() { cout << "Constructor de A" << endl; }
   void show() { cout << "Showing..." << endl; }
};

class B : virtual public A {
public:
   B() { cout << "Constructor de B" << endl; }
};

class C : virtual public A {
public:
   C() { cout << "Constructor de C" << endl; }
};
```

```
class D : public B, public C {
public:
   D() { cout << "Constructor de D" << endl; }
};

int main() {
   D d;
   // Hay dos caminos para llegar a show()
   d.show(); // No hay error

   return 0;
}
```

Imprime:

```
Constructor de A
Constructor de B
Constructor de C
Constructor de D
Showing...
```

Object Slicing: cuidado!

Ocurre cuando un objeto de una clase derivada se copia en una variable de la clase base por valor.

Esto suele hacerse cuando simplemente se quiere un objeto con la información definida en la clase base.

En ese proceso, la parte derivada se pierde, quedando solo la porción correspondiente a la clase base. Esto es un comportamiento válido del lenguaje.

En el ejemplo, no solo se puede perder el atributo “y”, sino que también se puede perder la posibilidad de invocar los métodos definidos en la clase derivada. Esto se debe a que no es posible acceder al comportamiento específico definido en la clase derivada.

```
#include <iostream>

class Base {
public:
   int x;
};

class Derivada : public Base {
public:
   int y;
};

int main() {
   Derivada d;
   d.x = 1; d.y = 2;

   // Slicing, se pierde y.
   Base b = d;
   return 0;
}
```

Tipos de Relación entre Objetos

Los tipos de relación son las formas en que las clases interactúan y se conectan entre sí para modelar relaciones. Hay 6 tipos:

1. Asociación
2. Agregación
3. Composición
4. Generalización
5. Realización
6. Dependencia

Estas relaciones representan conceptos léxicos como “is-a”, “has-a”, “uses-a” y “is-like-a”. Por ejemplo, “Un perro is-a mamífero”, “Un auto has-a motor”, etc. Estas relaciones no tienen que ser uno a uno.

Relaciones entre Objetos - Asociación

La asociación representa una relación estructural entre dos o más clases cuyos objetos están vinculados entre sí de manera persistente. Esto implica que la relación forma parte del estado de los objetos (por ejemplo, mediante atributos), a diferencia de una dependencia, que es típicamente temporal y se da durante la ejecución de una operación. Esta relación no implica, por sí misma, ownership entre los objetos ni dependencia en sus ciclos de vida.

Ejemplo: en un sistema de biblioteca, un Libro y un Autor están asociados. Un libro puede tener uno o más autores (por ejemplo, mediante referencias), y un autor puede haber escrito varios libros. Esta relación se mantiene en el tiempo como parte del modelo del sistema, independientemente de operaciones puntuales. Nótese que aquí el libro no está compuesto por autores, sino que mantiene una relación con ellos para acceder a su información.

En esta relación, los objetos pueden interactuar entre sí, pero sus ciclos de vida son independientes.

Relaciones entre Objetos - Agregación

La agregación representa una relación del tipo “has-a” entre un objeto que actúa como un todo (whole) y sus partes (parts). En este caso, el todo mantiene una relación con las partes, pero sin una dependencia total: el ciclo de vida de un objeto no depende del otro. Es decir, aunque los objetos están más estrechamente relacionados que en una simple asociación, las partes pueden existir de manera independiente del todo.

Ejemplo: profesores (*parts*) y un departamento de una universidad (*whole*). Los profesores pueden ser contratados, administrados económicamente, evaluados, organizados en roles, etc. En cuanto a su lifetime, en el caso de que departamento se cierre, los profesores pueden cambiar de departamento sin dejar de ser profesores. O bien, si renuncian todos los profesores, el departamento deberá de contratar más profesores, pero no dejar de existir.

En este caso hay una relación más intima entre los objetos dado que existe una acción entre el departamento y los profesores.

Relaciones entre Objetos - Composición

Composición representa una relación del tipo “has-a” que es más fuerte que agregación. En este caso, un objeto (whole) tiene ownership sobre otros (parts) y controla sus lifetimes (ciclos de vida). Esto significa que, cuando el objeto contenedor es destruido, sus partes (objetos contenidos) también son eliminadas.

Ejemplo: una casa (*whole*) y sus habitaciones (*parts*). Los cuartos por si solos no tienen sentidos fuera de una casa. Al mismo tiempo, si una casa es destruida, las habitaciones también son destruidas porque son una parte integral de la misma.

Relaciones entre Objetos - Generalización

Generalización es una relación “is-a” que permite crear, a partir de una clase más abstracta (llamada superclase o clase base), una clase más específica (llamada subclase o clase derivada). La subclase hereda propiedades y métodos de la superclase, los cuales pueden ser modificados y o bien pueden agregar nuevos.

Ejemplo: tener una clase mamífero que tenga atributos como cantidad de patas, desplazarse, etc. Clases derivadas de estas pueden ser un gato (“is-a” mamífero) y una ballena (“is-a” mamífero). Un objeto gato tiene 4 patas y se desplaza corriendo y el objeto ballena no tiene patas y se desplaza nadando.

Herencia y potencialmente polimorfismo permiten la implementación de esta relación.

Relaciones entre Objetos - Realización

Universidad de SanAndrés

Realización se refiere a la relación mediante la cual una clase concreta implementa el contrato definido por una interfaz (y, en muchos casos, por una clase abstracta). Es decir, la clase concreta proporciona la implementación de los métodos declarados en dicha abstracción.

Ejemplo: en un sistema se utilizan clases círculo y rectángulo que requieren la implementación de calcularArea() y calcularPerimetro(). Para asegurar su implementación se utiliza una interfaz o una clase abstracta. Como estas figuras tienen fórmulas distintas para estos métodos, ambas clases deberán proporcionar sus propias implementaciones de estos métodos.

Realización está fuertemente relacionada con polimorfismo (tema de la próxima presentación).

Relaciones entre Objetos - Dependencia

Dependencia es una relación en la cual una clase utiliza a otra para cumplir una función específica. Esta relación es generalmente temporal, ya que la clase dependiente no mantiene una referencia persistente al objeto utilizado, sino que lo emplea de forma puntual (por ejemplo, como parámetro o variable local). A diferencia de una asociación, esta relación no forma parte del estado del objeto, sino que se establece únicamente durante la ejecución de una operación.

Ejemplo: en un servicio de correo electrónico, una clase Usuario puede depender de una clase ServicioEmail para enviar un mensaje. En este caso, Usuario utiliza ServicioEmail sin necesidad de conocer su implementación interna. Además, ServicioEmail no forma parte del estado de Usuario, sino que es utilizado únicamente para ejecutar la operación de envío.

Tipos de Relaciones entre Objetos
Asociación
Agregación
Composición
Generalización
Realización
Dependencia
ASOCIACIÓN
ClaseA
ClaseB
Relación normal
AGREGACIÓN
ClaseA
ClaseB
Tiene un débil
COMPOSICIÓN
ClaseA
ClaseB
1
Parte de fuerte
DEPENDENCIA
ClaseX
ClaseY
Uso temporal
REALIZACIÓN
<Inteface>
ClaseC
Implementa interfaz
ClaseC
Implementa
GENERALIZACIÓN
ClaseSuper
ClaseSub
Es un

Preguntas?